Creative Software Design
3. Lab1
C++: iostream

Dept. of Computer Science

Hanyang University

Standard Input & Output

- C: **`printf(), scanf()`**
  - `#include <stdio.h>`
  - `scanf("%d", &num);`
  - `printf("hello %d\n", num);`

- C++: **`std::cout, std::cin`**, stream operators (>>, <<)
  - `#include <iostream>`
  - `std::cin >> num;`
  - `std::cout << "hello " << num << std::endl;`
  - This is the C++ way of input / output, but you can still use C-style input / output in your C++ code.

C++ iostream

- Stream: a sequence of bytes flowing in/out of the programs
  - **>>** - stream extraction operator: [stream] >> [variable]
  - **<<** - stream insertion operator: [stream] << [variable or value]
  - **std::cout** - standard output stream, normally the screen
  - **std::cin** - standard input stream, normally the keyboard
  - **std::endl** - inserts a newline character (**'\n'**)

```
#include <iostream>

int main()
{
    char s[10];
    int i;
    double d;
    std::cin >> s >> i >> d;
    std::cout << "s:" << s1 << ", i:" << i << ", d:" << d << std::endl;
    return 0;
}
```


std::cout

- Standard **output** stream
  - works exactly same as **printf(...)** in C
  - does not with a format specifier, i.e., **"%d"**
    - **"%x"** ➔ **std::hex**
    - **"%d"** ➔ **std::dec**
  - comes with **<<** (the stream insertion operator)
    - [stream] << [variable or value]
    - **std::endl** - inserts a newline character (**'\n'**)

std::cin

- Standard **input** stream
  - works exactly same as **`scanf(...)`** in C
  - comes with **`>>`** (the stream extraction operator)
    - [stream] >> [variable or value]
  - does not with a format specifier, i.e., **`"%d"`**
    - type-mismatched values ➔ fails

```
int x;
std::cin >> x;
if (std::cin.fail()) {
    std::cin.clear();
    char c;
    std::cin >> c;
}
```

```
int x;
if (!(std::cin >> x)) {
    std::cin.clear();
    char c;
    std::cin >> c;
}
```


1. Account Book - 1

- Come with a default budget of KRW 300,000
- You have two commands
  - e: expense
  - q: quit

  You would be broke when you spend more than your budget

```
Your Budget: 300000
e: expense, q: quit > e
New expense: 100000


Processing....


Your Budget: 200000
e: expense, q: quit > q
```

2. Account Book - 2

- Show the expense history
  - Manage the history up to 5 expenses
  - The program ends after 5 entries are entered

```
Your Budget:  260000
h: history e: expense, q: quit > h
[1] 1000
[2] 1000
[3] 35000
[4] 3000

Your Budget:  260000
h: history e: expense, q: quit >
```


2. Account Book – 2 (Cnt'd)

- **`std::setw()`** in **`iomanip`**
  - Print up to 7 digits with right justification
  - **`std::setw(int n)`**
  - **`std::right`**

```
Your Budget:  260000
h: history e: expense, q: quit > h
[ 1]    1000
[ 2]    1000
[ 3]   35000
[ 4]    3000

Your Budget:  260000
h: history e: expense, q: quit >
```

3. Simple ChatBot

- Write a simple chatbot that simply repeats what you say
  - **`std::getline(cin, str)`** to store the whole sentence

```
ITE1015 ChatBot
To quit the messenger, type "bye"!!
You> Hi
Bot> Hi
You> Hello, there
Bot> Hello, there
You> Bye
Bot> Bye
You> bye
```


4. Simple ChatBot - 2

- (bit) advanced chatbot that can recognize a set of commands
- "upper" – to convert the next sentence in uppercase
  - **`int std::toupper(int)`** can be used for this purpose

    @see https://en.cppreference.com/w/cpp/string/byte/toupper.html

```
My ChatBot
To quit the messenger, type "bye"!!
You> Hi
Bot> Hi
You> upper
You> Hello, there
Bot> HELLO, THERE
You> bye
```